

Question :

What is FPGA-based Edge Computing? Write its main advantages.

Answer :

1. FPGA-based Edge Computing is a way of doing things that puts together edge computing and special hardware called FPGA to make things faster. This means that of doing hard work on big computers far away we do it on smaller computers that are closer to our phones and other devices.
2. The good things about FPGA-based Edge Computing are:
3. It makes things happen faster because the computer is near us.
4. It uses power on our phones and the smaller computers.
5. It is really good at doing work for things like learning from pictures and understanding what we see.
6. We can make the FPGA hardware do lots of things at the time.
7. It makes things, like finding faces in pictures and recognizing objects more fun to use.

based Edge Computing-এর ধারণা ব্যাখ্যা কর এবং এর প্রধান সুবিধাগুলো উল্লেখ কর।

উত্তর

FPGA-ভিত্তিক এজ কম্পিউটিং (Edge Computing) হলো এমন একটি পদ্ধতি যা এজ কম্পিউটিং এবং FPGA নামক বিশেষ হার্ডওয়্যারকে একত্রিত করে কাজের গতি বৃদ্ধি করে। এর অর্থ হলো, দূরবর্তী কোনো বড় কম্পিউটারে জটিল কাজ করার পরিবর্তে আমরা সেই কাজগুলো আমাদের ফোন বা অন্যান্য ডিভাইসের কাছাকাছি থাকা ছোট কম্পিউটারে সম্পন্ন করি।

FPGA-ভিত্তিক এজ কম্পিউটিং-এর সুবিধাসমূহ হলো:

কম্পিউটারটি আমাদের কাছাকাছি থাকায় এটি কাজ দ্রুত সম্পন্ন করতে পারে। এটি আমাদের ফোন এবং ছোট কম্পিউটারগুলোর শক্তি বা বিদ্যুৎ ব্যবহার করে কাজ চালায়।

ছবি থেকে শেখা এবং দৃশ্যমান বিষয়বস্তু বোঝার মতো কাজগুলো এটি অত্যন্ত দক্ষতার সাথে করতে পারে।

আমরা FPGA হার্ডওয়্যারকে দিয়ে একই সময়ে একাধিক কাজ করতে পারি।
এটি ছবি থেকে মানুষের মুখ শনাক্ত করা বা বিভিন্ন বস্তু চিনে ফেলার মতো
কাজগুলোকে আরও কার্যকর করে তোলে।

Exam Question

Q. (5 marks) What is Processing-in-Memory (PIM)? Explain how it differs from the conventional von Neumann architecture, and discuss why it is needed for modern memory-intensive applications like DNNs.

Answer Outline

1. Definition of PIM (1 mark)

Processing-in-Memory (PIM) is an architectural approach in which computation is performed directly within or very close to the memory unit, rather than exclusively in a separate central processing unit. This reduces the need to continuously move data back and forth between the processor and memory.

2. Difference from Von Neumann Architecture (2 marks)

In the conventional Von Neumann architecture, the processor (CPU) and memory are physically separate units connected by a bus. Every computation requires data to be fetched from memory into the processor, processed, and then written back to memory. As memory sizes and workloads grow, this constant data movement across the bus becomes a major bottleneck — commonly known as the 'memory wall' — leading to limited bandwidth, higher latency, and increased energy consumption.

In contrast, PIM architectures place processing logic inside or adjacent to the memory itself (for example, within the logic layer of 3D-stacked memory). This drastically reduces the distance data must travel, thereby lowering latency and energy costs while enabling much higher effective memory bandwidth.

3. Why PIM is Needed for Modern Applications like DNNs (2 marks)

Modern workloads such as Deep Neural Networks (DNNs) are highly memory-intensive: they involve large-scale data reuse, huge numbers of parameters (weights), and frequent memory accesses. Research has shown that moving data

is often far more costly in terms of time and energy than performing the actual computation on it. Conventional architectures therefore struggle to deliver high performance and energy efficiency for such workloads due to the memory wall problem.

PIM addresses this by bringing computation closer to data, significantly reducing costly off-chip data transfers. This is especially effective when combined with technologies like 3D-stacked DRAM, which offers large memory capacity along with low-cost, high-bandwidth access — making PIM a promising solution for building DNN accelerators with better performance and energy efficiency.

বাংলা অনুবাদ

Exam Question

পরীক্ষার প্রশ্ন

Q. (5 marks) What is Processing-in-Memory (PIM)? Explain how it differs from the conventional von Neumann architecture, and discuss why it is needed for modern memory-intensive applications like DNNs.

প্রশ্ন: (৫ নম্বর) Processing-in-Memory (PIM) কী? এটি প্রচলিত von Neumann architecture থেকে কীভাবে ভিন্ন তা ব্যাখ্যা করো এবং DNN-এর মতো আধুনিক memory-intensive application-এর জন্য এটি কেন প্রয়োজন, তা আলোচনা করো।

Answer Outline

উত্তরের কাঠামো

1. Definition of PIM (1 mark)

১. PIM-এর সংজ্ঞা (১ নম্বর)

Processing-in-Memory (PIM) is an architectural approach in which computation is performed directly within or very close to the memory unit, rather than exclusively in a separate central processing unit. This reduces the need to continuously move data back and forth between the processor and memory.

Processing-in-Memory (PIM) হলো এমন একটি architectural approach যেখানে আলাদা central processing unit-এ computation করার পরিবর্তে সরাসরি memory unit-এর ভিতরে অথবা memory-এর খুব কাছাকাছি computation করা হয়। এর ফলে processor এবং memory-এর মধ্যে বারবার data আদান-প্রদান করার প্রয়োজন কমে যায়।

2. Difference from Von Neumann Architecture (2 marks)

২. Von Neumann Architecture থেকে পার্থক্য (২ নম্বর)

In the conventional Von Neumann architecture, the processor (CPU) and memory are physically separate units connected by a bus. Every computation requires data to be fetched from memory into the processor, processed, and then written back to memory. As memory sizes and workloads grow, this constant data movement across the bus becomes a major bottleneck — commonly known as the 'memory wall' — leading to limited bandwidth, higher latency, and increased energy consumption.

প্রচলিত Von Neumann architecture-এ processor (CPU) এবং memory আলাদা physical unit এবং একটি bus-এর মাধ্যমে সংযুক্ত থাকে। প্রতিটি computation-এর জন্য memory থেকে data processor-এ আনতে হয়, সেখানে processing করতে হয় এবং পরে আবার memory-তে লিখে দিতে হয়। Memory-এর size এবং workload বাড়ার সঙ্গে সঙ্গে bus-এর মাধ্যমে এই ক্রমাগত data movement একটি বড় bottleneck হয়ে দাঁড়ায়, যা সাধারণত 'memory wall' নামে পরিচিত। এর ফলে bandwidth কমে যায়, latency বেড়ে যায় এবং energy consumption বৃদ্ধি পায়।

In contrast, PIM architectures place processing logic inside or adjacent to the memory itself (for example, within the logic layer of 3D-stacked memory). This drastically reduces the distance data must travel, thereby lowering latency and energy costs while enabling much higher effective memory bandwidth.

এর বিপরীতে, PIM architecture-এ processing logic memory-এর ভিতরে অথবা memory-এর একেবারে কাছাকাছি স্থাপন করা হয় (যেমন, 3D-stacked memory-এর logic layer-এর মধ্যে)। এর ফলে data-কে যে দূরত্ব অতিক্রম করতে হয় তা অনেক কমে যায়। তাই latency এবং energy cost কমে এবং অনেক বেশি effective memory bandwidth পাওয়া যায়।

3. Why PIM is Needed for Modern Applications like DNNs (2 marks)

৩. DNN-এর মতো আধুনিক Application-এর জন্য PIM কেন প্রয়োজন (২ নম্বর)

Modern workloads such as Deep Neural Networks (DNNs) are highly memory-intensive: they involve large-scale data reuse, huge numbers of parameters (weights), and frequent memory accesses. Research has shown that moving data is often far more costly in terms of time and energy than performing the actual computation on it. Conventional architectures

therefore struggle to deliver high performance and energy efficiency for such workloads due to the memory wall problem.

Deep Neural Networks (DNNs)-এর মতো আধুনিক workload অত্যন্ত memory-intensive। এতে large-scale data reuse, বিপুল সংখ্যক parameters (weights) এবং ঘন ঘন memory access প্রয়োজন হয়। Research থেকে দেখা গেছে, কোনো data-এর উপর actual computation করার চেয়ে data move করতেই অনেক বেশি time এবং energy খরচ হতে পারে। তাই memory wall problem-এর কারণে conventional architecture এই ধরনের workload-এর ক্ষেত্রে high performance এবং energy efficiency দিতে সমস্যায় পড়ে।

PIM addresses this by bringing computation closer to data, significantly reducing costly off-chip data transfers. This is especially effective when combined with technologies like 3D-stacked DRAM, which offers large memory capacity along with low-cost, high-bandwidth access — making PIM a promising solution for building DNN accelerators with better performance and energy efficiency.

PIM computation-কে data-এর আরও কাছে নিয়ে এসে এই সমস্যার সমাধান করে এবং ব্যয়বহুল off-chip data transfer উল্লেখযোগ্যভাবে কমিয়ে দেয়। 3D-stacked DRAM-এর মতো technology-এর সঙ্গে এটি বিশেষভাবে কার্যকর, কারণ এটি বড় memory capacity-এর পাশাপাশি low-cost, high-bandwidth access প্রদান করে। ফলে ভালো performance এবং energy efficiency-সহ DNN accelerator তৈরির জন্য PIM একটি promising solution।

Efficient Mitchell's Approximate Log Multipliers for Convolutional Neural Networks

Min Soo Kim, Alberto A. Del Barrio, Leonardo Tavares Oliveira, Roman Hermida, ' Senior Member, IEEE, Nader Bagherzadeh, Fellow, IEEE.

Describe the working principle of the proposed approximate logarithmic multiplier and explain how it achieves energy efficiency in CNN applications.

Answer: The proposed approximate logarithmic multiplier is based on Mitchell's logarithmic multiplication algorithm. It converts multiplication into logarithm addition, which reduces the hardware complexity compared with conventional multipliers. To further reduce hardware cost, the design uses w-truncation, improved Leading-One Detector (LOD), optimized shift calculations, and simplified decoder and zero-detection units. These techniques reduce the number of hardware operations and consequently lower power and energy consumption. The multiplier is mainly applied during CNN inference, where approximate computation can provide significant resource savings with only a small accuracy degradation.

For example, at $w=8$, the proposed multiplier achieves about 88% less energy consumption than an exact 32-bit fixed-point multiplier, with only 0.2% performance degradation on the ILSVRC2012 validation dataset.

Therefore, the proposed multiplier achieves energy-efficient CNN inference by trading a small amount of computational accuracy for significant hardware and energy savings.

প্রস্তাবিত Approximate Logarithmic Multiplier এর কার্যপ্রণালী বর্ণনা কর এবং CNN applications এ এটি কীভাবে energy efficiency অর্জন করে তা ব্যাখ্যা কর।

প্রস্তাবিত Approximate Logarithmic Multiplier মূলত Mitchell's Logarithmic Multiplication Algorithm এর উপর ভিত্তি করে তৈরি। এখানে সরাসরি multiplication করার পরিবর্তে multiplication কে logarithm এর addition এ রূপান্তর করা হয়। এর ফলে প্রচলিত multiplier এর তুলনায় hardware complexity কমে যায়।

Hardware cost আরও কমানোর জন্য এই design এ w-truncation, উন্নত Leading One Detector (LOD), optimized shift calculation এবং simplified decoder ও zero detection unit ব্যবহার করা হয়েছে। এর মাধ্যমে hardware operations এর সংখ্যা কমে যায় এবং ফলে power ও energy consumption কমে।

এই multiplier মূলত CNN inference এর সময় ব্যবহার করা হয়। CNN inference এ approximate computation ব্যবহার করলে সামান্য performance loss এর বিনিময়ে অনেক hardware resource ও energy সাশ্রয় করা সম্ভব।

উদাহরণস্বরূপ, $w=8$ হলে proposed multiplier একটি exact 32-bit fixed-point multiplier এর তুলনায় প্রায় 88% কম energy ব্যবহার করে। এর বিনিময়ে

ILSVRC2012 validation dataset এ মাত্র 0.2% performance degradation দেখা যায়।

Question: Explain the major contributions of the NeuRRAM RRAM-based Compute-in-Memory (CIM) chip. How does it improve energy efficiency, flexibility, and inference accuracy?

Answer: NeuRRAM is a 48-core RRAM based Compute-in-Memory (CIM) chip that accelerates AI computing with high energy efficiency, flexibility and accuracy.

1. Energy efficiency: We present an RRAM crossbar based CIM that stores weights and performs computation in memory to minimize energy consumption in data movement.
2. Flexibility: The novel TNSA (Transposable Neurosynaptic Array) architecture allows dataflow permutations for different kinds of networks including CNN, LSTM and RBM.
3. Large-scale parallelism: The chip consists of 48 CIM cores which can perform inference in parallel and also adopt various weight-mapping schemes.
4. Efficient neuron circuit: A simple but effective voltage-mode neuron circuit is utilized which can be controlled for computation bit precision with only minor impact on power and area.
5. High-accuracy: Through algorithm-hardware co-design, the hardware-aware software optimizations suppress the impact of hardware non-idealities, achieving high accuracy comparable to software implementations. For example, NeuRRAM achieved 99.0% accuracy on MNIST, 85.7% on CIFAR-10, and 84.7% on Google Speech Command.

NeuRRAM হলো একটি 48-কোর RRAM-ভিত্তিক Compute-in-Memory (CIM) চিপ, যা উচ্চ শক্তি-দক্ষতা, নমনীয়তা এবং নির্ভুলতার মাধ্যমে AI কম্পিউটিংকে দ্রুততর করে।

1. শক্তি-দক্ষতা (Energy Efficiency): RRAM crossbar-ভিত্তিক CIM প্রযুক্তিতে AI মডেলের weight সংরক্ষণ করা হয় এবং memory-এর মধ্যেই

computation সম্পন্ন করা হয়। ফলে memory ও processor-এর মধ্যে ডেটা স্থানান্তরের জন্য প্রয়োজনীয় শক্তি কমে যায়।

2. **নমনীয়তা (Flexibility):** নতুন TNSA (Transposable Neurosynaptic Array) architecture বিভিন্ন ধরনের neural network, যেমন CNN, LSTM এবং RBM, অনুযায়ী dataflow পরিবর্তন বা পুনর্বিন্যাস করতে পারে।
3. **বৃহৎ-পরিসরের Parallelism:** চিপটিতে 48টি CIM core রয়েছে, যা একসাথে parallel inference করতে পারে এবং বিভিন্ন ধরনের weight-mapping scheme ব্যবহার করতে পারে।
4. **দক্ষ Neuron Circuit:** একটি সহজ কিন্তু কার্যকর voltage-mode neuron circuit ব্যবহার করা হয়েছে। এটি computation-এর bit precision নিয়ন্ত্রণ করতে পারে, যেখানে power consumption ও chip area-তে খুব সামান্য প্রভাব পড়ে।
5. **উচ্চ নির্ভুলতা (High Accuracy):** Algorithm-hardware co-design এবং hardware-aware software optimization ব্যবহার করে hardware-এর non-idealities-এর প্রভাব কমানো হয়েছে। ফলে software implementation-এর কাছাকাছি উচ্চ accuracy অর্জন করা সম্ভব হয়েছে। উদাহরণস্বরূপ, NeuRRAM MNIST-এ 99.0%, CIFAR-10-এ 85.7% এবং Google Speech Command-এ 84.7% accuracy অর্জন করেছে।

Q. Explain NoCarry and NoCarry+ approximate adder algorithms. How do they reduce energy consumption and computation steps compared with exact IMPLY-based adders?

Answer

The two approximate algorithms called **NoCarry** and **NoCarry+** for IMPLY-based memristive adders.

1. NoCarry Algorithm:

In the NoCarry approach, carry propagation is omitted from the approximated bits. Since carry calculation requires additional IMPLY operations, removing it significantly reduces the number of computation steps. The output Sum is still calculated and stored in the b-memristor.

2. NoCarry+ Algorithm:

NoCarry+ is an improved version of NoCarry. Instead of completely ignoring

carry, it calculates the **Carry-out only at the last approximated bit**. This improves the error behavior while still avoiding carry propagation through the earlier approximated bits.

3. Main Advantage:

By reducing carry-related IMPLY operations, both algorithms require fewer computation steps and consume less energy than the corresponding exact adders. The paper reports **up to 54% energy savings** compared with the corresponding exact algorithms.

4. Trade-off:

The reduction in operations introduces some approximation error. However, the error is considered acceptable for **error-tolerant applications such as image processing**. NoCarry+ generally provides better error behavior than NoCarry because it retains the final carry calculation.

NoCarry এবং NoCarry+ Approximate Adder Algorithm ব্যাখ্যা করো। Exact IMPLY-based Adder-এর তুলনায় এগুলো কীভাবে Energy Consumption এবং Computation Steps কমায়?

উত্তর

Paper-এ IMPLY-based memristive adder-এর জন্য NoCarry এবং NoCarry+ নামে দুটি Approximate Algorithm প্রস্তাব করা হয়েছে।

১. NoCarry Algorithm:

NoCarry পদ্ধতিতে Approximate করা Bit-গুলোর জন্য Carry Propagation বাদ দেওয়া হয়। যেহেতু Carry Calculation-এর জন্য অতিরিক্ত IMPLY Operation প্রয়োজন হয়, তাই Carry বাদ দিলে মোট Computation Step উল্লেখযোগ্যভাবে কমে যায়। তবে Sum হিসাব করা হয় এবং সেটি b-memristor-এ সংরক্ষণ করা হয়।

২. NoCarry+ Algorithm:

NoCarry+ হলো NoCarry Algorithm-এর একটি উন্নত সংস্করণ। এখানে সম্পূর্ণভাবে Carry বাদ না দিয়ে শুধুমাত্র শেষ Approximate Bit-এ Carry-out হিসাব করা হয়। এর ফলে আগের Approximate Bit-গুলোর মধ্যে Carry Propagation-এর প্রয়োজন হয় না এবং একইসঙ্গে Error কিছুটা কমানো যায়।

৩. প্রধান সুবিধা:

Carry-সম্পর্কিত IMPLY Operation কমানোর মাধ্যমে উভয় Algorithm-ই Exact Adder-এর তুলনায় কম Computation Step ব্যবহার করে এবং কম Energy খরচ করে। Paper-এ দেখানো হয়েছে যে সংশ্লিষ্ট Exact Algorithm-এর তুলনায় সর্বোচ্চ 54% পর্যন্ত Energy Saving করা সম্ভব।

৪. Trade-off বা বিনিময়:

Computation Step এবং Energy কমানোর বিনিময়ে কিছু Approximation Error তৈরি হয়। তবে এই Error Image Processing-এর মতো Error-Tolerant Application-এর জন্য গ্রহণযোগ্য বলে বিবেচিত হয়। NoCarry+ সাধারণত NoCarry-এর তুলনায় ভালো Error Performance দেয়, কারণ এতে শেষ Carry Calculation রাখা হয়েছে।

Loop Tiling and Choosing the Tile Size

Question

A single layer of a neural network is much bigger than the on-chip memory of an FPGA, so an accelerator has to work through it in blocks. The compiler settles on a block size before any hardware exists, and for an ordinary layer there are thousands of sizes it could choose.

- (a) What is loop tiling, and why do we need it? [2/]**
- (b) A compiler cannot try every tile size by building the hardware for each one. Explain why not, say what it does instead, and give one thing that can go wrong. [3/]**

Answer

- (a)** Tiling splits a large layer into blocks small enough to sit in on-chip memory. One block comes in from external DRAM, the accelerator finishes everything that block needs, the result goes back out, and only then does the next one arrive. We tile because the layer will not fit. Without it a value would have to be fetched from DRAM every time it is used, and DRAM is slow next to the arithmetic, so the multipliers would spend most of their time waiting.
- (b)** Building the hardware for a single tile size means high-level synthesis followed by place-and-route. Ten minutes would be optimistic; an hour or two is normal. With a few thousand candidates to work through, that search never finishes. So compilers estimate instead: a short formula predicts the runtime of each candidate at almost no cost, which lets the tool score thousands of them and then look properly at the handful that come out on top. The catch is that a formula only knows what its author put in it. Build one from the arithmetic alone and it will score two tile sizes the same even though the first keeps the multipliers fed while the second leaves them stalled on memory. Whatever it names as best should be checked against a real measurement at least once.

বিষয়

একটা নিউরাল নেটওয়ার্কের একটা লেয়ারের পুরো ডেটা FPGA-র ভেতরের মেমোরিতে একসাথে ধরে না। তাই অ্যাক্সিলারেটর পুরো লেয়ার একবারে না নিয়ে ছোট ছোট ব্লকে ভাগ করে কাজ করে। এই ভাগ করে নেওয়ার নামই লুপ টাইলিং, আর ব্লকের আকারটাকে বলা হয় টাইল সাইজ। মুশকিল হলো, হার্ডওয়ার বানানোর আগেই কম্পাইলারকে এই আকার ঠিক করে ফেলতে হয়, অথচ বেছে নেওয়ার মতো আকার থাকে হাজার হাজার।

প্রশ্ন

প্রথম অংশে জানতে চাওয়া হয়েছে, লুপ টাইলিং জিনিসটা আসলে কী আর কেন সেটা লাগে। দ্বিতীয় অংশে জানতে চাওয়া হয়েছে, প্রতিটা সম্ভাব্য টাইল সাইজের জন্য আলাদা করে হার্ডওয়ার বানিয়ে দেখা কেন সম্ভব না, কম্পাইলার তার বদলে কী করে, আর সেই পদ্ধতিতে কোথায় গণ্ডগোল হতে পারে।

উত্তর

(ক) টাইলিং মানে বড় লেয়ারটাকে এমন ছোট ছোট ব্লকে ভাগ করা, যেগুলো অন-চিপ মেমোরিতে ধরে। বাইরের DRAM থেকে একটা ব্লক ভেতরে আসে, সেই ব্লকের যা যা হিসাব দরকার সব শেষ হয়, ফলাফল আবার বাইরে চলে যায়, তারপর আসে পরের ব্লক। টাইলিং করা হয় কারণ পুরো লেয়ার ভেতরে ধরে না। না করলে প্রতিটা মান যতবার লাগত ততবারই DRAM থেকে আনতে হতো, আর DRAM যেহেতু হিসাবের তুলনায় ধীর, গুণক ইউনিটগুলো বেশির ভাগ সময় বসেই থাকত।

(খ) একটা মাত্র টাইল সাইজের জন্য হার্ডওয়ার বানাতে গেলে high-level synthesis, তারপর place-and-route চালাতে হয়। দশ মিনিটে হয়ে যাবে ভাবলে ভুল হবে, এক-দুই ঘণ্টা লাগাই স্বাভাবিক। কয়েক হাজার সম্ভাব্য আকারের জন্য এটা করতে গেলে কাজ আর শেষই হবে না। তাই কম্পাইলার অনুমানের পথ ধরে: ছোট একটা সূত্র দিয়ে প্রতিটা আকারের সম্ভাব্য সময় বের করে ফেলা যায় প্রায় বিনা খরচে, ফলে হাজার হাজার বিকল্পের নম্বর বের করে শুধু সেরা কয়েকটাকে ভালো করে দেখা সম্ভব হয়। ঝামেলাটা হলো, সূত্র ততটুকুই জানে যতটুকু তাতে ঢোকানো হয়েছে। শুধু গণনার হিসাব ধরে সূত্র বানাতে সেটা দুটো টাইল সাইজকে সমান নম্বর দেবে, যদিও একটা গুণক ইউনিটগুলোকে ব্যস্ত রাখে আর অন্যটা সেগুলোকে মেমোরির অপেক্ষায় বসিয়ে রাখে। তাই সূত্র

যেটাকে সেরা বলছে, অন্তত একবার সত্যিকারের হার্ডওয়্যারে মেপে দেখে নেওয়া উচিত।

English Question:

Why do we need a completely new hardware design like PEDSA when we already have pretty fast AI chips?

Answer: Think about how a regular AI chip would work. It's like a chef who keeps going to the fridge for each ingredient. It's like a chef who keeps going to the fridge for each ingredient. It's a huge waste of time. In hardware, that “fridge” is the external memory. In hardware, that “fridge” is the external memory. Saving and fetching data back and forth creates a traffic jam. PEDSA solves this by creating a continuous assembly line on the counter top. The data flows straight through from the beginning to the end. Can't stop. No going back; no return.

And that's why it can do big things like image segmentation so damn fast. And that's why it can do big things like image segmentation so damn fast.

Bangla Question:

আমাদের তো এমনিতে বেশ ভালো AI প্রসেসর আছেই, তাহলে PEDSA-র মতো একেবারে নতুন একটা হার্ডওয়্যার ডিজাইন করার দরকার পড়লো কেন?

Answer: সাধারণ প্রসেসরগুলো কীভাবে কাজ করে একটু ভেবে দেখুন। ওগুলো অনেকটা এমন রাঁধুনির মতো, যে এক চিমটি লবণ আনার জন্যও বারবার ফ্রিজের কাছে ছুটে যায়। এতে প্রচুর সময় নষ্ট হয়! প্রসেসরের ভাষায় এই "ফ্রিজ" হলো তার এক্সটার্নাল মেমোরি। বারবার মেমোরিতে ডেটা রাখতে আর আনতে গেলে সিস্টেমে বিশাল ট্রাফিক জ্যাম তৈরি হয়। PEDSA এই সমস্যাটাই সলভ করেছে একটা টানা অ্যাসেম্বলি লাইন তৈরি করে। এখানে একবার ডেটা ঢুকলে কাজ শেষ না হওয়া পর্যন্ত কোথাও থামে না। বারবার মেমোরিতে ফিরে যাওয়ার কোনো ঝামেলা নেই। আর ঠিক এ কারণেই ইমেজ সেগমেন্টেশনের মতো ভারী কাজেও এটা এতো ফাস্ট পারফর্ম করে।

Question

Explain the major challenges of Design Space Exploration (DSE) for large-scale HLS designs and discuss how AutoScaleDSE addresses these challenges to achieve scalable and efficient design exploration.

Answer

Introduction:

Design Space Exploration (DSE) for large-scale High-Level Synthesis (HLS) designs is challenging because the number of possible design configurations increases significantly with the size and complexity of the design. A major limitation is the sub-design merging problem, where independently optimized sub-designs cannot always be combined successfully because of resource sharing and incompatibility between their configurations.

1. Function Call Graph:

AutoScaleDSE first analyzes the source design and generates a function call graph containing information about the hierarchical structure, computational intensity, and dependencies among sub-designs.

2. ROI Partitioning:

The large design is divided into different Regions of Interest (ROIs) containing closely related sub-designs. Computationally intensive sub-designs are selected as seeds, and the regions are expanded toward highly correlated sub-designs. This makes the merging process more suitable for large designs.

3. Random Forest Classifier:

A pre-trained Random Forest classifier is used to predict whether different sub-design configurations are compatible. It reduces the number of invalid and non-Pareto-optimal design points without requiring every combination to be individually evaluated by the HLS compiler. The classifier achieved approximately 97% accuracy.

4. Genetic Algorithm:

When the theoretically estimated performance differs significantly from the HLS-evaluated result, AutoScaleDSE applies a Genetic Algorithm (GA) to improve the selected design point. GA helps find configurations that are more compatible with the HLS compiler, particularly for shared memory elements.

5. Final Design Construction:

The process is repeated for different ROIs, and the best candidates are finally combined to construct the overall design. In this way, AutoScaleDSE can

explore large-scale designs using a top-down, code-structure-aware approach while reducing the expensive HLS compilation required during exploration.

Conclusion:

Therefore, AutoScaleDSE improves the scalability of HLS DSE by combining ROI partitioning, Random Forest-based compatibility prediction, HLS-based correction, and Genetic Algorithm optimization.

বাংলায় ব্যাখ্যা

প্রশ্নের বাংলা অর্থ:

বড় আকারের HLS design-এর Design Space Exploration (DSE)-এর প্রধান সমস্যাগুলো কী এবং AutoScaleDSE কীভাবে এই সমস্যাগুলো সমাধান করে scalable ও efficient design exploration অর্জন করে—ব্যাখ্যা কর।

ভূমিকা:

বড় HLS design-এর ক্ষেত্রে অনেকগুলো possible configuration বা design point থাকে। Design-এর size ও complexity যত বাড়ে, এই design point-এর সংখ্যাও তত বাড়ে। ফলে প্রতিটি design point-কে আলাদাভাবে HLS compiler দিয়ে পরীক্ষা করা অনেক সময়সাপেক্ষ হয়ে যায়।

১. Function Call Graph:

প্রথমে design-এর প্রয়োজনীয় information সংগ্রহ করে একটি Function Call Graph তৈরি করা হয়। এতে design-এর hierarchy, computational intensity এবং sub-design-এর dependency সম্পর্কে তথ্য থাকে।

২. ROI Partitioning:

বড় design-টিকে কয়েকটি Region of Interest (ROI)-তে ভাগ করা হয়। যেসব sub-design একে অপরের সাথে বেশি related, সেগুলোকে একই ROI-তে রাখা হয়। এতে unrelated sub-design ভুলভাবে merge হওয়ার সম্ভাবনা কমে যায়।

৩. Random Forest:

Random Forest classifier ব্যবহার করে আগে থেকেই predict করা হয় কোন sub-design configuration compatible এবং কোনটি incompatible। এর ফলে প্রতিটি combination-এর জন্য HLS compiler চালাতে হয় না। Paper-এ classifier প্রায় 97% accuracy অর্জন করেছে।

৪. Genetic Algorithm:

কখনও theoretical result এবং HLS compiler-এর actual result-এর মধ্যে পার্থক্য দেখা যায়। তখন Genetic Algorithm ব্যবহার করে design point-টিকে আরও

optimize করা হয়। বিশেষ করে shared memory elements-এর configuration উন্নত করতে এটি সাহায্য করে।

৫. Final Design:

প্রতিটি ROI-এর জন্য ভালো design candidate নির্বাচন করার পর সেগুলো merge করে final overall design তৈরি করা হয়। এভাবে AutoScaleDSE বড় ও complex HLS design-এর DSE-কে আরও scalable এবং efficient করে।

সহজে মনে রাখার উপায়:

Function Call Graph → ROI Partitioning → Random Forest → HLS Evaluation → Genetic Algorithm → Final Design

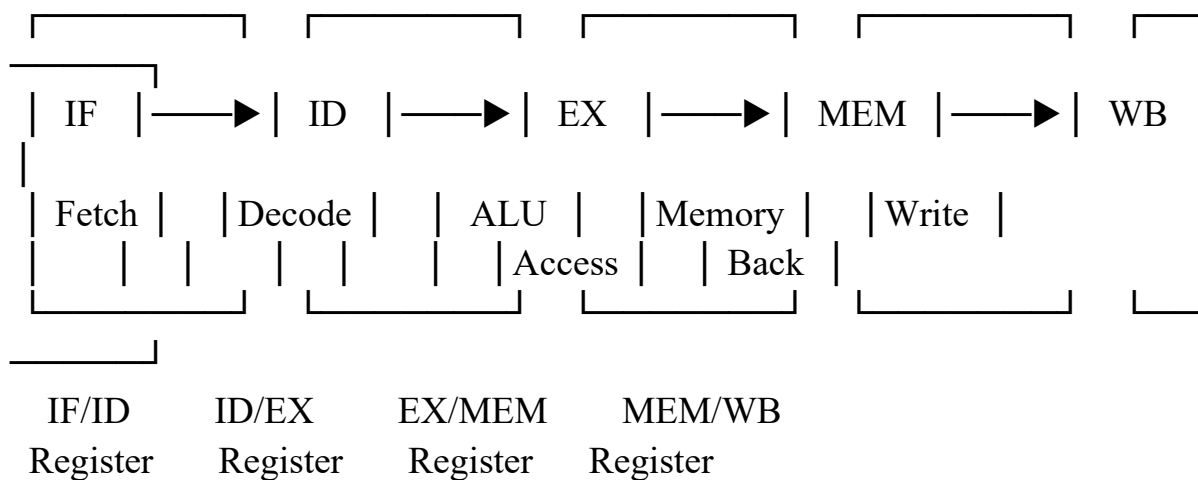
Question

Explain the working principle of a five-stage pipelined processor based on RISC-V. Describe the function of each stage with a suitable block diagram.

Answer

A five-stage RISC-V pipelined processor divides instruction execution into five stages: **Instruction Fetch (IF)**, **Instruction Decode (ID)**, **Execute (EX)**, **Memory Access (MEM)**, and **Write Back (WB)**. Pipeline registers are used between the stages so that several instructions can be processed simultaneously. This overlapping operation improves instruction throughput and processor performance.

Block Diagram



1. Instruction Fetch (IF): The PC provides the address of the next instruction. The instruction memory fetches it and sends it to the IF/ID pipeline register.

2. Instruction Decode (ID): The instruction is decoded, operands are read from the register file, and the required control signals are generated. The result is passed to the ID/EX register.

3. Execute (EX): The ALU performs the required arithmetic or logical operation and passes the result to the EX/MEM register.

4. Memory Access (MEM): The processor reads data from or writes data to the data memory according to the instruction.

5. Write Back (WB): The ALU result or memory data is written back to the appropriate register, completing the instruction execution.

Conclusion: The five-stage pipeline allows different instructions to be processed in different stages at the same time. Therefore, it improves instruction throughput and overall processor performance.

বাংলা অনুবাদ

Five-stage RISC-V pipeline-এ instruction execution পাঁচটি ধাপে সম্পন্ন হয়: **IF, ID, EX, MEM এবং WB**। প্রতিটি stage-এর মাঝে pipeline register থাকে, যার ফলে একাধিক instruction একই সময়ে বিভিন্ন stage-এ কাজ করতে পারে। এতে instruction throughput এবং processor performance বৃদ্ধি পায়।

IF: PC পরবর্তী instruction-এর address দেয় এবং instruction memory থেকে instruction সংগ্রহ করে।

ID: Instruction decode করা হয়, register থেকে operand নেওয়া হয় এবং control signal তৈরি করা হয়।

EX: ALU প্রয়োজনীয় arithmetic বা logical operation সম্পন্ন করে।

MEM: প্রয়োজন অনুযায়ী data memory থেকে data read অথবা memory-তে data write করা হয়।

WB: ALU result অথবা memory data নির্দিষ্ট register-এ write back করা হয়।

সারসংক্ষেপ: পাঁচটি stage-এ instruction processing overlap করার কারণে processor-এর throughput ও performance উন্নত হয়।

Paper: Heterogeneous Confidential Computing System for Large Language Models: A Survey

Question (5 Marks): What is a Trusted Execution Environment (TEE), and why is it important for securing LLMs in heterogeneous computing systems?

Answer (English):

A Trusted Execution Environment (TEE) is a hardware-enforced isolated environment that protects sensitive code and data even from privileged software such as the operating system or hypervisor. TEEs are important for LLM security because modern LLM systems use heterogeneous hardware, such as CPUs, GPUs, and NPUs, and sensitive data moves between these components. The paper explains that traditional CPU-based TEEs, such as Intel SGX, AMD SEV, and Arm TrustZone, protect CPU-side computation, but critical accelerator computation may remain unprotected. Therefore, heterogeneous TEEs aim to extend confidentiality and integrity protection to accelerators. They help protect model weights, user prompts, KV-cache, activations, and other sensitive data from attacks such as data leakage, cross-tenant attacks, and untrusted software. TEEs also use mechanisms such as secure I/O, tenant isolation, and remote attestation to build trusted execution paths. Thus, TEEs provide an important balance between security and performance for secure LLM execution.

উত্তর (বাংলা):

Trusted Execution Environment (TEE) হলো হার্ডওয়্যার দ্বারা নিয়ন্ত্রিত একটি নিরাপদ ও বিচ্ছিন্ন পরিবেশ, যেখানে সংবেদনশীল কোড ও ডেটা Operating System বা Hypervisor-এর মতো privileged software থেকেও সুরক্ষিত থাকে। LLM নিরাপত্তার জন্য TEE গুরুত্বপূর্ণ, কারণ আধুনিক LLM সিস্টেমে CPU, GPU এবং NPU-এর মতো বিভিন্ন ধরনের হার্ডওয়্যার একসাথে ব্যবহার করা হয় এবং এই উপাদানগুলোর মধ্যে সংবেদনশীল ডেটা আদান-প্রদান হয়।

পেপারটিতে বলা হয়েছে যে Intel SGX, AMD SEV এবং Arm TrustZone-এর মতো প্রচলিত CPU-based TEE মূলত CPU-এর computation সুরক্ষিত করে, কিন্তু accelerator-এর গুরুত্বপূর্ণ computation অনেক সময় সুরক্ষার বাইরে থাকে। তাই heterogeneous TEE-এর লক্ষ্য হলো GPU ও NPU-এর মতো accelerator-কেও

confidentiality এবং integrity protection-এর মধ্যে আনা। এটি model weights, user prompts, KV-cache, activations এবং অন্যান্য sensitive data-কে data leakage, cross-tenant attack এবং untrusted software থেকে রক্ষা করতে সাহায্য করে। Secure I/O, tenant isolation এবং remote attestation-এর মতো প্রযুক্তি ব্যবহার করে TEE একটি trusted execution path তৈরি করে। তাই LLM-এর নিরাপত্তা ও কর্মক্ষমতার মধ্যে ভারসাম্য বজায় রাখতে TEE একটি গুরুত্বপূর্ণ প্রযুক্তি।

Paper Reference: Hao Lan et al., “Heterogeneous Confidential Computing System for Large Language Models: A Survey,” ACM Transactions on Architecture and Code Optimization, 2026.

3006

IEEE TRANSACTIONS ON COMPUTER-AIDED DESIGN OF INTEGRATED CIRCUITS AND SYSTEMS, VOL. 44, NO. 8, AUGUST 2025

PBS: Program Behavior-Aware Scheduling for High-Level Synthesis

Aoxiang Qin[✉], *Member, IEEE*, Rongjie Yang, Minghua Shen[✉], *Member, IEEE*,
and Nong Xiao[✉], *Senior Member, IEEE*

Question:

What is the main objective of the PBS scheduling method? How does it improve HLS scheduling?

Answer:

The main objective of **PBS (Program Behavior-Aware Scheduling)** is to improve scheduling in **High-Level Synthesis (HLS)** by considering both **operation dependency behavior** and **resource requirement behavior**. It's goal is to simultaneously reduce **latency** and **resource usage** while finding high-quality scheduling solutions.

PBS improves **HLS scheduling** in several ways. It uses **Hybrid State Encoding (HSE)** to represent both program behavior. It uses a **Bi-directional Graph Neural Network** to learn operation dependencies from both forward and backward directions. It also uses **Multiresolution Aggregation** to capture global

features of the Data Flow Graph (DFG). Finally, these features are integrated with **Reinforcement Learning (RL)** to iteratively improve the initial schedule. Thus, PBS can search the scheduling solution space more effectively and achieve better scheduling performance.

PBS (Program Behavior-Aware Scheduling)-এর প্রধান উদ্দেশ্য কী? এটি কীভাবে HLS scheduling কে ধাপে ধাপে উন্নত করে?

উত্তর: PBS (Program Behavior-Aware Scheduling)-এর প্রধান উদ্দেশ্য হলো **High-Level Synthesis (HLS)**-এ scheduling উন্নত করা। এজন্য এটি একই সাথে **operation dependency behavior** এবং **resource requirement behavior** বিবেচনা করে। এর মূল লক্ষ্য হলো **latency** এবং **resource usage** উভয়ই কমিয়ে উন্নত scheduling solution তৈরি করা।

PBS বিভিন্ন উপায়ে HLS scheduling উন্নত করে। এটি **Hybrid State Encoding (HSE)** ব্যবহার করে program-এর উভয় ধরনের behavior প্রকাশ করে। **Bi-directional Graph Neural Network** ব্যবহার করে **operation dependency**-কে forward এবং backward—দুই দিক থেকে শেখে। **Multiresolution Aggregation** ব্যবহার করে পুরো **Data Flow Graph**-এর global features সংগ্রহ করে। তারপর এসব features-কে **Reinforcement Learning** -এর সাথে ব্যবহার করে schedule-কে ধাপে ধাপে উন্নত করা হয়। ফলে PBS scheduling solution space আরও কার্যকরভাবে অনুসন্ধান করতে পারে এবং সবচেয়ে ভালো scheduling performance অর্জন করে।

Question:

What are the major challenges of using SRAM-based Computing-in-Memory (CIM) for practical-scale neural network inference? How can a heterogeneous SRAM–nonvolatile memory architecture help address these challenges? Discuss briefly.

Answer

The main idea behind SRAM-CIM is to perform the calculation close to the memory to avoid the data transaction between memory and processor again and again. As a result speed is improved and it becomes energy efficient.

But the problem is, big neural networks need to store huge amount of weights. As SRAM cells are comparatively bigger so it's difficult to store data in a chip.

As a result data needs to be brought from external memory which increases the latency and not energy efficient. This problem is called memory wall problem. Another problem is reliability. Specially for device and manufacturing variation, stored data cannot be restored properly in some cases. Moreover, increasing storage density can be reason of low accuracy.

That's where heterogeneous SRAM-NVM architecture can be useful.

Here alongside SRAM, nonvolatile memories like ReRAM are used. ReRam can store more data in short spaces. That's why more Neural Network weights can be stored inside the chip. On the other side, SRAM is used for computation purposes.

Inclusion of hybrid coding improves the efficiency of storing the weights. So that memory requirement reduces but the neural network accuracy remains as it is. Hardware utilization and throughput can be increased by using data mapping and optimized computing techniques.

Therefore, there are three main advantages of heterogeneous SRAM-NVM CIM, they are: storage density, reliability, accuracy and energy efficiency. They can bring a proper balance in computing by enhancing performance. That's why in for practical-scale neural network's on chip inference, it is more efficient.

বাংলা:

SRAM-CIM-এর মূল idea হলো memory-এর কাছেই computation করা, যাতে বারবার processor আর memory-এর মধ্যে data আনা-নেওয়া করতে না হয়। এতে speed বাড়ে এবং energy কম লাগে।

কিন্তু সমস্যা হচ্ছে, বড় neural network-এর জন্য অনেক অনেক weight store করতে হয়। SRAM cell তুলনামূলক বড় হওয়ায় chip-এর মধ্যে বেশি data রাখা যায় না। ফলে কিছু data বাইরের memory থেকে আনতে হয়। এতে আবার energy এবং latency বেড়ে যায়। এটাকে সহজভাবে memory wall problem বলা যায়।

আরেকটা সমস্যা হলো reliability। বিশেষ করে device বা manufacturing variation-এর কারণে stored data ঠিকভাবে restore নাও হতে পারে। আবার storage density বাড়াতে গিয়ে যদি accuracy কমে যায়, তাহলে সেটাও সমস্যা।

এখানেই heterogeneous SRAM-NVM architecture কাজে আসে।

এখানে SRAM-এর সাথে ReRAM-এর মতো nonvolatile memory ব্যবহার করা হয়। ReRAM বেশি data কম জায়গায় store করতে পারে। ফলে neural network-এর অনেক বেশি weight chip-এর ভেতরেই রাখা সম্ভব হয়। অন্যদিকে SRAM computation-এর জন্য ব্যবহার করা যায়।

এর সাথে hybrid coding ব্যবহার করলে weight-গুলো আরও efficiently store করা যায়। ফলে memory requirement কমে, কিন্তু neural network-এর accuracy খুব বেশি নষ্ট হয় না। Data mapping এবং optimized computing techniques ব্যবহার করলে hardware utilization এবং throughput-ও বাড়ানো যায়।

সুতরাং, heterogeneous SRAM–NVM CIM-এর মূল সুবিধা হলো এটি storage density, reliability, accuracy, energy efficiency এবং computing performance-এর মধ্যে একটি ভালো balance তৈরি করতে পারে। এ কারণে practical-scale neural network-এর on-chip inference-এর জন্য এটি বেশি উপযোগী।

Paper Name: “FINN: A Framework for Fast, Scalable Binarized Neural Network Inference” (ID-2102005)

Question: What is FINN & how does it improve the performance of Binarized Neural Network (BNN) inference on FPGAs?

Answer:

FINN is a framework for building fast, flexible & scalable FPGA accelerator for Binarized Neural Networks (BNNs). It uses a heterogeneous streaming architecture where a separate compute engine is assigned to each neural network layer. Since BNN weights & activation use 1-bit values, FINN uses efficient hardware operations such as XNOR and popcount instead of conventional multiplication and accumulation. It also converts batch normalization and activation into a simple threshold operation and implements max-pooling using a Boolean OR operation. The network parameters are kept in on-chip memory, reducing off-chip memory access and latency. FINN also uses configurable processing element and SIMD lanes to adjust the hardware according to the required throughput. As a result, FINN achieves very high classification throughput with low latency and power consumption.

প্রশ্ন: FINN কী এবং এটি কীভাবে FPGA-তে Binarized Neural Network (BNN) inference এর performance উন্নত করে ?

উত্তর:

FINN হলো Binarized Neural Network (BNN)-এর জন্য দ্রুত, flexible এবং scalable FPGA accelerator তৈরি করার একটি framework। এটি heterogeneous streaming architecture ব্যবহার করে, যেখানে neural network এর প্রতিটি Layer এর জন্য আলাদা compute engine থাকে। BNN এর weight ও activation 1-bit হওয়ায় FINN প্রচলিত multiplication ও accumulation এর পরিবর্তে XNOR এবং pop count এর মতো Efficient hardware operation ব্যবহার করে। এছাড়া batch normalization ও activation-কে একটি সহজ threshold operation এবং max-pooling-কে Boolean OR operation হিসেবে বাস্তবায়ন করা হয়। Network-এর Parameters on-chip Memory তে রাখা হয়, ফলে Off-chip memory access কমে এবং latency হ্রাস পায়। এছাড়া Configurable processing elements ও SIMD lanes ব্যবহার করে প্রয়োজন অনুযায়ী throughput নিয়ন্ত্রণ করা যায়। ফলে FINN খুব বেশি Classification throughput, কম Latency এবং কম Power consumption অর্জন করতে পারে।

What is SCALE-Sim v3, and what are the five major enhancements introduced over SCALE-Sim v2?

SCALE-Sim v3 is a cycle-accurate simulator for systolic computing hardware accelerators employed in performing deep learning jobs. It allows researchers to explore the performance of a neural network on accelerator hardware by forecasting some parameters, such as cycles of execution, memory activity, utilization, latency, as well as energy consumption and power use.

There are five notable improvements from SCALE-Sim v2 to SCALE-Sim v3:

1. Multi-core capability

The new version of SCALE-Sim can model the operations of several cores/tensor cores operating simultaneously, providing multi-core design as well as shared memory and burden distribution.

2. Sparsity support

The new generation of this simulator can deal with sparse workloads. Specifically, it allows one to implement the two following types of sparsity:

Layer-wise sparsity – that is, each layer works with different sparsity.

Row-wise sparsity – that is, the sparsity level is defined for each row of data using combination N:M.

3. Main memory integration

The previous version of SCALE-Sim supposed that all processes had been carried out using on-chip memory. The new version integrates Ramulator model to provide realistic memory behavior and take into account its activity, delays, performance.

4. DATA LAYOUT MODELING OF SCALE-Sim V3

SCALE-Sim v3 describes the physical structure of data inside SRAMs while taking into consideration the bank conflicts, which occur when two or more processing entities request to use the same memory bank at the same time, thus causing delays. This provides a more true-to-life prediction of system performance.

5. ENERGY AND POWER MODELING

SCALE-Sim v3 works with Accelergy to obtain estimations of the energy and power used by an accelerator. To achieve this, SCALE-Sim calculates the number of operations performed, such as MAC operations and memory accesses, and uses this data in analysis made by Accelergy

SCALE-Sim v3 কী এবং SCALE-Sim v2-এর তুলনায় এর ৫টি প্রধান উন্নতি কী?

SCALE-Sim v3 হলো একটি cycle-accurate simulator, যা deep learning কাজের জন্য ব্যবহৃত systolic-array hardware accelerator-এর performance পরীক্ষা করতে ব্যবহার করা হয়।

সহজভাবে বললে, এটি গবেষকদের বুঝতে সাহায্য করে যে কোনো neural network accelerator hardware-এ চালালে কেমন performance দেবে। এটি execution cycle, memory activity, utilization, latency, energy consumption এবং power consumption ইত্যাদি অনুমান করতে পারে। SCALE-Sim v2-এর তুলনায় SCALE-Sim v3-তে ৫টি গুরুত্বপূর্ণ উন্নতি আনা হয়েছে:

1. Multi-Core Capability

নতুন SCALE-Sim একসাথে একাধিক **core** বা **tensor core**-এর কাজ **simulate** করতে পারে। এতে: Multiple cores ব্যবহার করা যায়, Shared memory model করা যায়, কাজগুলো বিভিন্ন core-এর মধ্যে ভাগ করা যায়

অর্থাৎ, এটি **multi-core accelerator**-এর কাজ আরও বাস্তবভাবে **simulate** করতে পারে।

2. Sparsity Support

SCALE-Sim v3 **sparse workload** support করে। **Sparsity** বলতে বোঝায় **data**-এর অনেক **value zero** বা খুব ছোট হওয়া। এখানে দুই ধরনের **sparsity support** করা হয়:

Layer-wise sparsity: Neural network-এর প্রতিটি **layer**-এ আলাদা **sparsity level** থাকতে পারে।

Row-wise sparsity: প্রতিটি **row**-এর জন্য **sparsity** নির্ধারণ করা যায় **N:M ratio** ব্যবহার করে।

অর্থাৎ, SCALE-Sim v3 বাস্তব neural network-এর **sparse data pattern** আরও ভালোভাবে **simulate** করতে পারে।

3. Main Memory Integration

SCALE-Sim v2 ধরে নিত যে প্রয়োজনীয় **data on-chip memory**-তেই **available** আছে। কিন্তু বাস্তবে **data main memory/DRAM** থেকেও আসতে পারে এবং সেখানে **delay** ও **memory stall** হতে পারে। তাই SCALE-Sim v3-তে **Ramulator memory model** যুক্ত করা হয়েছে।

এটি: **Memory latency**, **Memory performance**, **DRAM behavior**, **Memory stalls**, **Bank conflicts** ইত্যাদি আরও বাস্তবভাবে **model** করতে পারে।

4. Data Layout Modeling

SCALE-Sim v3 **SRAM**-এর মধ্যে **data** কীভাবে সাজানো থাকে সেটিও **model** করে।

এখানে **bank conflict** বিবেচনা করা হয়। **Bank conflict** হলো যখন একাধিক **processing unit** একই সময়ে একই **memory bank** থেকে **data** চাইলে, তাদের অপেক্ষা করতে হয় এবং ফলে **delay** তৈরি হয়। তাই এই **feature accelerator**-এর বাস্তব **performance** আরও **accurately estimate** করতে সাহায্য করে।

5. Energy and Power Modeling

SCALE-Sim v3 **Accelergy**-এর সাথে কাজ করে accelerator-এর **energy** এবং **power consumption** estimate করতে পারে।SCALE-Sim বিভিন্ন operation-এর সংখ্যা হিসাব করে, যেমন:MAC operations,Memory accessesতারপর এই information **Accelergy** ব্যবহার করে energy analysis করা হয়।

Q: What is the R-chain in ChainPIM and how does it enable the accelerator to process Heterogeneous Graph Neural Networks (HGNNs) at the same time with both high parallelism and vertex data reuse? Describe in detail with a diagram.

Answer

The key new idea behind ChainPIM is R-chain, a chain-structure-based mechanism which achieves both parallelism and vertex data reuse, both of which are mutually exclusive in most cases.

How it is constructed :

- 1) The idea of ChainPIM stems from Routine: Radix sort. Unlike sorting numbers according to their digit position, it sorts metapaths according to the type of their vertex (in the successive key positions along the path).
- 2) The vertex types at a position are grouped together when there are more than one metapaths; the splitting is repeated n times (repetition factor), thus creating groups.
- 3) In each group, metapaths with common vertices are connected to form a chain structure: the R-chain.

How it helps :

- (1) **Vertex Reuse:** In these R chains vertices end up getting reused when instances share the same overlapped metapath prefixes. Once the data for

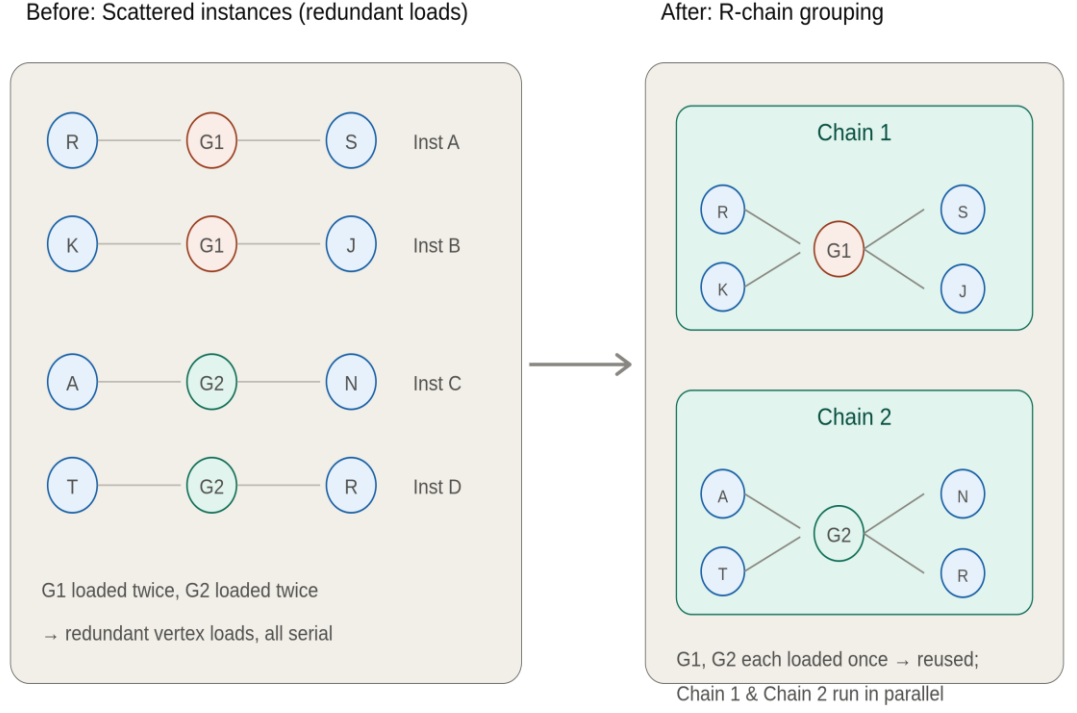
a shared vertex is loaded into the ReRAM crossbar it can just stay there and get used for every instance in the chain.

That seems to cut down on loading the same vertex multiple times.

Previous ReRAM PIM designs spent most of their inference time on structural and semantic aggregation, probably over ninety percent of it. I think this reuse is what makes the difference but it might depend on how much overlap actually shows up.

(2) **Parallelism:** R chains do not share vertices at all so they stay independent from one another. That lets them run on separate ReRAM tiles or crossbars without getting in each others way. It seems this setup catches instance level parallelism and metapath level parallelism at the same time. Traditional designs usually stick to vertex or feature level stuff and end up missing that. I think the gain comes from handling the chains together but I am not totally sure how the crossbars end up managing the load in practice.

(3) **Trade-off control:** The balance is tuned by the repetition factor n -- smaller n favors reuse (fewer, larger chains), larger n favors parallelism (more, smaller chains). Experiments show that $n=1$ achieves the best overall performance with 47.86% average memory reduction and $128.29\times$ speedup over GPU.



সংক্ষেপে ডায়াগ্রামটা যা দেখাচ্ছে: বাম দিকে ৪টা আলাদা instance-এ G1 আর G2 নোড দুইবার করে বারবার লোড হচ্ছিল (redundant, serial)। ডান দিকে R-chain গ্রুপিং-এর পর, একই নোড শেয়ার করা instance-গুলো এক চেইনে বাঁধা পড়েছে — G1/G2 এখন একবারই লোড হয় (reuse), আর Chain 1 ও Chain 2 একে অপরের থেকে স্বাধীন হওয়ায় সমান্তরালে চলতে পারে (parallelism)।

R-chain হলো ChainPIM-এর মূল নতুন অবদান — এটি একটি চেইন-ভিত্তিক কাঠামো, যেটি সম্পর্কিত metapath instance-গুলোকে একসাথে সাজিয়ে

একইসাথে **parallelism** এবং **vertex data reuse** — এই দুটি সাধারণত পরস্পরবিরোধী লক্ষ্য একসাথে অর্জন করে।

কীভাবে তৈরি হয় :

- ChainPIM Radix sort-এর অনুপ্রেরণায় তৈরি। সংখ্যা সাজানোর পরিবর্তে, এটি metapath-এর পথ ধরে ক্রমিক "key position"-এ থাকা vertex type অনুযায়ী metapath-গুলোকে গ্রুপ করে।
- একই position-এ একই vertex type থাকা metapath-গুলো একই গ্রুপে পড়ে; এই ভাগ করাটা n বার (*repetition factor*) পুনরাবৃত্তি করা হয়, যা আলাদা আলাদা গ্রুপ তৈরি করে।
- প্রতিটা গ্রুপের মধ্যে, যেসব metapath নোড শেয়ার করে, তাদের একটা chain structure-এ মেলানো হয় — এটাই R-chain।

এটা কীভাবে সাহায্য করে :

1. **Vertex Reuse:** একই R-chain-এর ভেতরের instance-গুলো কমন নোড শেয়ার করে (metapath-এর ওভারল্যাপিং prefix)। একবার শেয়ার্ড নোডের ডেটা ReRAM crossbar-এ লোড হলে, সেই চেইনের সব instance সেটা reuse করে — আগের ডিজাইনে যে redundant vertex loading সমস্যা ছিল (যার কারণে >90% সময় structural+semantic aggregation-এ চলে যেত), সেটা দূর হয়।
2. **Parallelism:** আলাদা আলাদা R-chain একে অপরের থেকে স্বাধীন (কোনো নোড শেয়ার করে না), তাই এগুলো বিভিন্ন ReRAM tile/crossbar-এ সমান্তরালে চালানো যায় — instance-level ও metapath-level parallelism কাজে লাগায়, যা আগের vertex/feature-level parallelism-নির্ভর ডিজাইনগুলো পারত না।
3. **Trade-off নিয়ন্ত্রণ:** Repetition factor n এই ভারসাম্য ঠিক করে — ছোট n reuse-এর দিকে ঝোঁকে (কম, বড় চেইন), বড় n parallelism-এর দিকে ঝোঁকে (বেশি, ছোট চেইন)। experiment-এ দেখা গেছে $n=1$ সবচেয়ে ভালো ফলাফল দেয় — গড়ে 47.86% মেমরি কমায় এবং GPU-এর তুলনায় $128.29\times$ গতি বাড়ায়।

Question: Explain the concept of "asymmetric fault tolerance" in scheduler synthesis and how it helps manage hardware overhead. (5 Marks)

Asymmetric fault tolerance refers to an unequal rule applied to scheduler behavior in the presence and absence of data hazards.

When a hazard exists → the scheduler must stall. This is mandatory and non-negotiable, since proceeding would produce an incorrect result.

When no hazard exists → the scheduler is still permitted to stall, even though it isn't strictly required to. This may cause a minor loss in performance, but it never compromises correctness.

How It Helps Manage Hardware Overhead

Building hardware that can perfectly detect every rare hazard-free opportunity requires extensive logic circuitry, which drives up chip area, power consumption, and timing complexity. Asymmetric fault tolerance keeps these costs under control in three main ways:

1. Simplifies Control Logic

If identifying a rare "safe-to-issue" case would require disproportionately large or complex circuitry, the designer can skip building that logic entirely. The scheduler simply defaults to a stall signal for that case instead — trading a small, occasional performance loss for a significant reduction in hardware complexity.

2. Enables Smart Synthesis Optimizations

During hardware generation using tools such as Binary Speculation Diagrams (BSDs), synthesis algorithms apply techniques like dynamic pruning and early termination. These methods intentionally cut away logic branches that only yield marginal speed benefits, keeping the resulting circuit lightweight and efficient.

3. Guarantees Functional and Compact Hardware

By prioritizing correctness over chasing every possible performance gain, engineers ensure the synthesized scheduler is always functionally correct (no hazard is ever missed), while also fitting within strict constraints on chip area, power budget, and signal timing.

প্রশ্ন: শিডিউলার সিন্থেসিসে "Asymmetric Fault Tolerance"-এর ধারণাটি ব্যাখ্যা করুন এবং এটি কীভাবে হার্ডওয়্যার ওভারহেড কমাতে সাহায্য করে তা লিখুন। (৫ নম্বর)

অ্যাসিমেট্রিক ফল্ট টলারেন্স (Asymmetric Fault Tolerance) হলো ডেটা হ্যাজার্ড (Data Hazard) থাকা এবং না থাকার পরিস্থিতিতে শিডিউলারের আচরণের একটি অসমান নিয়ম।

যখন হ্যাজার্ড থাকে → শিডিউলারকে অবশ্যই থামতে হবে (Stall)। এটি বাধ্যতামূলক, কারণ তা না করলে সিস্টেমে ভুল আউটপুট তৈরি হবে।

যখন হ্যাজার্ড থাকে না → শিডিউলার চাইলে তখনও থামতে পারে, যদিও তখন থামাটা জরুরি নয়। এর ফলে পারফরম্যান্স কিছুটা কমেতে পারে, তবে সিস্টেমের সঠিকতায় কোনো প্রভাব পড়ে না।

কীভাবে এটি হার্ডওয়্যার ওভারহেড নিয়ন্ত্রণে সাহায্য করে

এমন হার্ডওয়্যার তৈরি করা যা প্রতিটি বিরল হ্যাজার্ড-মুক্ত সুযোগ নিখুঁতভাবে শনাক্ত করতে পারে, তার জন্য অনেক বেশি লজিক সার্কিটের প্রয়োজন হয় — যা চিপের আকার, বিদ্যুৎ খরচ, এবং টাইমিং জটিলতা বাড়িয়ে দেয়। অ্যাসিমেট্রিক ফল্ট টলারেন্স তিনটি উপায়ে এই খরচ নিয়ন্ত্রণে রাখে:

১. কন্ট্রোল লজিক সহজ করে

যদি কোনো বিরল "নিরাপদ অবস্থা" শনাক্ত করতে অতিরিক্ত বড় বা জটিল সার্কিটের প্রয়োজন হয়, তবে ডিজাইনার সেই জটিল লজিক তৈরি না করেই বাদ দিতে পারেন। এর পরিবর্তে শিডিউলার সেই ক্ষেত্রে সরাসরি একটি স্টল সিগন্যাল পাঠিয়ে দেয় — সামান্য পারফরম্যান্স কমিয়ে বড় ধরনের হার্ডওয়্যার জটিলতা এড়ানো হয়।

২. স্মার্ট সিন্থেসিস অপটিমাইজেশন সম্ভব করে

বাইনারি স্পেকুলেশন ডায়াগ্রাম (BSD)-এর মতো টুল ব্যবহার করে হার্ডওয়্যার তৈরির সময়, সিন্থেসিস অ্যালগরিদমগুলো ডায়নামিক প্রুনিং (Dynamic Pruning) এবং আর্লি টার্মিনেশন (Early Termination)-এর মতো কৌশল ব্যবহার করে। এই পদ্ধতিগুলো ইচ্ছাকৃতভাবে সেই লজিক ব্রাঞ্চগুলো ছেঁটে ফেলে যেগুলো থেকে খুবই সামান্য গতি সুবিধা পাওয়া যায়, ফলে সার্কিটটি হালকা ও কার্যকর থাকে।

৩. কার্যকর ও কমপ্যাক্ট হার্ডওয়্যারের নিশ্চয়তা দেয়

প্রতিটি সম্ভাব্য পারফরম্যান্স সুবিধার পেছনে না ছুটে সঠিকতাকে প্রাধান্য দেওয়ার মাধ্যমে প্রকৌশলীরা নিশ্চিত করতে পারেন যে সিন্থেসাইজড শিডিউলারটি সবসময় সম্পূর্ণ কার্যকর (কোনো হ্যাজার্ড কখনো মিস হয় না), এবং একইসাথে চিপের আকার, পাওয়ার বাজেট, ও সিগন্যাল টাইমিং-এর কঠোর সীমার মধ্যেই ফিট করে।

Question: A proposed approach addresses the challenges of high computation, pipeline blocking, and memory overhead in Transformer inference on edge devices. What are the main contributions of this approach, and how do they improve inference efficiency and scalability?

Answer:

The proposed solution has a number of contributions that can enhance the efficiency and scalability of Transformer inference on edge devices:

1. **Max-Finding-Free SA-Softmax** The authors use a threshold-based saturation approximation to replace the softmax max-finding operation. This eliminates the need to scan and buffer the whole row of attention before moving on. The result is less temporal dependency and pipeline blocking caused by max-finding.
2. **Softmax and Layer Fusion** The proposed design fuses the Softmax operation with the following $X \times V$ matrix multiplication. The full intermediate attention matrix is not stored and is not read/written to memory repeatedly; instead, the attention operations are performed together. This minimizes data transfer and memory access between layers.
3. **Dense-Sparse Hybrid Architecture** The architecture divides attention values into dense and sparse paths based on the saturation criterion. These paths are efficiently processed by dedicated hardware like a MAC array, Sparse Engine, and Post Processing Unit (PPU) prior to post-processing.
4. **Fine-Grained Sequence Parallelism** The authors distribute the sequence computation over multiple Transformer Processing Elements (TrPEs). This eliminates the max-finding dependency, meaning different parts of the sequence can be processed without waiting for the attention row to be

completed. This enhances the utilization and scalability of the processing elements.

প্রস্তাবিত সমাধানটি Edge Device-এ Transformer inference-এর efficiency এবং scalability বাড়ানোর জন্য কয়েকটি গুরুত্বপূর্ণ contribution করেছে:

1. **Max-Finding-Free SA-Softmax:**

Authors traditional Softmax-এর max-finding operation-এর পরিবর্তে threshold-based saturation approximation ব্যবহার করেছেন। ফলে পুরো attention row scan ও buffer করার প্রয়োজন কমে যায়। এর ফলে temporal dependency এবং pipeline blocking কমে।

2. **Softmax ও Layer Fusion:**

Proposed design-এ Softmax এবং পরবর্তী $X \times V$ matrix multiplication একসাথে execute করা হয়েছে। ফলে পুরো intermediate attention matrix বারবার memory-তে store ও read/write করতে হয় না। এতে data movement এবং memory access কমে যায়।

3. **Dense-Sparse Hybrid Architecture:**

Saturation criterion অনুযায়ী attention values-কে dense এবং sparse path-এ ভাগ করা হয়েছে। এরপর MAC Array, Sparse Engine এবং Post Processing Unit (PPU)-এর মতো dedicated hardware ব্যবহার করে এই দুই path efficiently process করা হয়েছে।

4. **Fine-Grained Sequence Parallelism:**

Sequence computation-কে একাধিক Transformer Processing Elements (TrPEs)-এর মধ্যে ভাগ করা হয়েছে। Max-finding dependency না থাকায় sequence-এর বিভিন্ন অংশ একসাথে process করা যায়। ফলে PE utilization এবং scalability বৃদ্ধি পায়।

1. **Why can a CNN architecture with lower FLOPs/MACs still perform poorly on real hardware? Explain how the Once-for-All (OFA) framework helps in hardware-aware CNN deployment, and discuss**

how input resolution, model depth, width, and kernel size affect performance across CPU, GPU, and AI accelerators.

Answer :

A CNN architecture with lower FLOPs/MACs does not always provide better real-world performance because FLOPs/MACs only represent computational complexity. They do not fully consider **memory bandwidth, memory access patterns, kernel availability, software stack, hardware utilization, and hardware-specific optimizations**. Therefore, a theoretically efficient model may still have high latency on a particular device.

The **Once-for-All (OFA)** framework addresses this problem by training a single super-network that contains many possible sub-networks. Different sub-networks can be generated by changing **input resolution, depth, width, and kernel size** without retraining the model. This allows the same architecture space to be evaluated on different hardware platforms.

The major observations are:

1. **Input Resolution:** Performance generally improves as resolution increases up to **224×224**, where hardware utilization reaches a high level.
2. **Model Depth:** Increasing depth can improve hardware utilization, particularly on GPU and Neural Engine. The ANE showed a large efficiency improvement when depth increased from 10 to 20 blocks.
3. **Model Width:** Increasing width does not benefit every device equally. ANE performance improved with larger expansion ratios, whereas excessive width reduced GPU performance because of increased resource and memory pressure.
4. **Kernel Size:** CPUs favor smaller **3×3 kernels**, while GPUs can benefit from larger kernels such as **7×7** because of their high parallel computational capability. The ANE showed relatively consistent performance across kernel sizes.

Thus, **there is no universally optimal CNN architecture**. The best architecture should be selected according to the target hardware rather than FLOPs/MACs alone.

বাংলা

1. “কোনো CNN architecture-এর FLOPs/MACs কম হওয়া সত্ত্বেও কেন সেটি বাস্তব Hardware-এ খারাপ Performance দিতে পারে? Hardware-aware CNN deployment-এ Once-for-All (OFA) framework

কীভাবে সাহায্য করে এবং CPU, GPU ও AI Accelerator-এ Input Resolution, Model Depth, Width এবং Kernel Size কীভাবে Performance-কে প্রভাবিত করে—ব্যাখ্যা কর।”

উত্তর

কোনো CNN architecture-এর FLOPs/MACs কম হলেই যে বাস্তব Hardware-এ তার Performance ভালো হবে—এটি সবসময় সত্য নয়। কারণ FLOPs/MACs মূলত Computational Complexity নির্দেশ করে; কিন্তু Memory Bandwidth, Memory Access Pattern, Kernel Availability, Software Stack এবং Hardware Utilization-এর মতো System-level বিষয়গুলো বিবেচনা করে না। ফলে কম FLOPs-এর একটি Model-ও নির্দিষ্ট Hardware-এ বেশি Latency দিতে পারে।

এই সমস্যা সমাধানের জন্য paper-এ Once-for-All (OFA) framework ব্যবহার করা হয়েছে। OFA একটি Super-network train করে, যার মধ্যে অনেকগুলো সম্ভাব্য Sub-network থাকে। Input Resolution, Depth, Width এবং Kernel Size পরিবর্তন করে বিভিন্ন Sub-network তৈরি করা যায় এবং নতুন করে Training না করেই সেগুলো বিভিন্ন Hardware-এ পরীক্ষা করা যায়।

প্রধান ফলাফলগুলো হলো—

১. **Input Resolution:**
Resolution বাড়ানোর সাথে Performance সাধারণত বৃদ্ধি পায় এবং 224×224 resolution-এ Hardware Utilization খুব ভালো পাওয়া যায়।
২. **Model Depth:**
Depth বাড়ালে Hardware Utilization বৃদ্ধি পেতে পারে। বিশেষ করে GPU ও Neural Engine-এ বেশি Depth থেকে ভালো Efficiency পাওয়া গেছে। ANE-তে Depth 10 থেকে 20 করলে Efficiency উল্লেখযোগ্যভাবে বৃদ্ধি পেয়েছে।
৩. **Model Width:**
Width বাড়ানোর প্রভাব Hardware অনুযায়ী ভিন্ন। ANE-তে বড় Expansion Ratio Performance বাড়িয়েছে। কিন্তু GPU-তে অতিরিক্ত Width Performance কমিয়েছে, কারণ এতে Register Pressure ও Memory Pressure বৃদ্ধি পেয়েছে।
৪. **Kernel Size:**
CPU সাধারণত ছোট 3×3 Kernel-এ ভালো Performance দেয়। অন্যদিকে GPU বড় 7×7 Kernel থেকে বেশি সুবিধা পেতে পারে, কারণ GPU-এর Parallel

Computing Capability বেশি। ANE-তে বিভিন্ন Kernel Size-এ Performance তুলনামূলকভাবে স্থিতিশীল ছিল।

সুতরাং, কোনো একটি CNN architecture সব Hardware-এর জন্য সর্বোত্তম নয়। Target Hardware অনুযায়ী Resolution, Depth, Width এবং Kernel Size নির্বাচন করতে হবে। শুধুমাত্র FLOPs/MACs দেখে CNN Architecture নির্বাচন করা উচিত নয়।

Computing Schemes in SRAM-Based Computing-in-Memory (CIM)

Question: (5 marks)

SRAM-based Computing-in-Memory (CIM) macros can be classified based on where computation is physically performed relative to the memory array. Briefly explain the different computing schemes used in SRAM-CIM.

Answer:

1. **Near-Memory-Array Computing (NMAC):** The compute circuit is placed adjacent to (outside) the memory array. Data is first read out from the memory cells, and computation (digital or analog) is then performed by the nearby circuit block within the same access cycle.
2. **In-Memory-Array Computing (IMAC):** Computation is performed directly inside the memory array itself. Each SRAM cell takes part in the computation (e.g., multiplication) via the analog bitline voltage/current during the memory access cycle.
3. **Non-local computing:** Standard, unmodified 6T/8T SRAM cells are used. Computation results (e.g., MAC) are obtained by combining bitline signals using peripheral circuits, without adding extra transistors inside the cell.
4. **In-array local computing (IA-LC):** One local computing cell (LCC) is shared by a group of neighboring SRAM cells to accumulate partial results locally within the array, balancing added area against computing capability.
5. **In-cell local computing (IC-LC):** Extra transistors are added inside every individual SRAM bitcell to perform computation (e.g., XNOR) locally. This gives the highest computing parallelism/complexity but the largest per-cell area overhead.

বাংলায়

প্রশ্ন: SRAM-ভিত্তিক Computing-in-Memory (CIM) ম্যাক্রোগুলোকে মেমরি অ্যারের সাপেক্ষে গণনা কোথায় সম্পাদিত হয় তার ভিত্তিতে শ্রেণীবদ্ধ করা যায়। SRAM-CIM-এ ব্যবহৃত বিভিন্ন কম্পিউটিং স্কিমগুলো সংক্ষেপে ব্যাখ্যা করো।

উত্তর:

6. **Near-Memory-Array Computing (NMAC):** কম্পিউটিং সার্কিট মেমরি অ্যারের পাশে (বাইরে) স্থাপন করা হয়। মেমরি সেল থেকে প্রথমে ডেটা রিড করা হয়, তারপর একই অ্যাক্সেস সাইকেলে পার্শ্ববর্তী সার্কিটে (ডিজিটাল বা অ্যানালগ) গণনা করা হয়।
7. **In-Memory-Array Computing (IMAC):** গণনা সরাসরি মেমরি অ্যারের ভেতরেই সম্পন্ন হয়। প্রতিটি SRAM সেল মেমরি অ্যাক্সেসের সময় অ্যানালগ bitline voltage/current-এর মাধ্যমে গণনায় (যেমন multiplication) অংশগ্রহণ করে।
8. **Non-local computing:** কোনো পরিবর্তন ছাড়াই সাধারণ 6T/8T SRAM সেল ব্যবহার করা হয়। সেলের ভেতরে অতিরিক্ত ট্রানজিস্টর যোগ না করেই, পেরিফেরাল সার্কিট bitline সিগন্যাল একত্রিত করে গণনার ফলাফল (যেমন MAC) বের করে।
9. **In-array local computing (IA-LC):** একটি local computing cell (LCC) কয়েকটি প্রতিবেশী SRAM সেলের একটি গ্রুপের মধ্যে শেয়ার করা হয়, যা অ্যারের ভেতরেই আংশিক ফলাফল যোগ করে — এতে যুক্ত এরিয়া ও কম্পিউটিং সক্ষমতার মধ্যে ভারসাম্য বজায় থাকে।
10. **In-cell local computing (IC-LC):** প্রতিটি পৃথক SRAM বিটসেলের ভেতরে অতিরিক্ত ট্রানজিস্টর যোগ করে স্থানীয়ভাবে গণনা (যেমন XNOR) করা হয়। এতে সর্বোচ্চ প্যারালালিজম ও জটিলতা পাওয়া যায়, তবে প্রতি সেলে এরিয়া ওভারহেড সবচেয়ে বেশি হয়।